time domain. Imagine that the frequency spectrum in (f) is compressed much harder, resulting in the time domain signal in (e) expanding into neighboring periods.

A second issue is to define exactly what it means to compress or expand a discrete signal. As shown in Fig. 10-12a, a discrete signal is compressed by compressing the underlying *continuous* curve that the samples lie on, and then resampling the new continuous curve to find the new discrete signal. Likewise, this same process for the expansion of discrete signals is shown in (e). When a discrete signal is compressed, events in the signal (such as the width of the pulse) happen over a *fewer* number of samples. Likewise, events in an expanded signal happen over a *greater* number of samples.

An equivalent way of looking at this procedure is to keep the underlying continuous waveform the same, but resample it at a different sampling rate. For instance, look at Fig. 10-13a, a discrete Gaussian waveform composed of 50 samples. In (b), the same underlying curve is represented by 400 samples. The change between (a) and (b) can be viewed in two ways: (1) the sampling rate has been kept constant, but the underlying waveform has been expanded to be eight times wider, or (2) the underlying waveform has been kept constant, but the sampling rate has increased by a factor of eight. Methods for changing the sampling rate in this way are called **multirate** techniques. If more samples are added, it is called **interpolation**. If fewer samples are used to represent the signal, it is called **decimation**. Chapter 3 describes how multirate techniques are used in ADC and DAC.

Here is the problem: if we are given an arbitrary discrete signal, how do we know what the underlying continuous curve is? It depends on if the signal's information is encoded in the *time domain* or in the *frequency domain*. For time domain encoded signals, we want the underlying continuous waveform to be a smooth curve that passes through all the samples. In the simplest case, we might draw straight lines between the points and then round the rough corners. The next level of sophistication is to use a curve fitting algorithm, such as a spline function or polynomial fit. There is not a single "correct" answer to this problem. This approach is based on minimizing irregularities in the *time domain* waveform, and completely ignores the frequency domain.

When a signal has information encoded in the frequency domain, we ignore the time domain waveform and concentrate on the frequency spectrum. As discussed in the last chapter, a finer sampling of a frequency spectrum (more samples between frequency 0 and 0.5) can be obtained by padding the time domain signal with zeros before taking the DFT. Duality allows this to work in the opposite direction. If we want a finer sampling in the time domain (interpolation), pad the frequency spectrum with zeros before taking the Inverse DFT. Say we want to interpolate a 50 sample signal into a 400 sample signal. It's done like this: (1) Take the 50 samples and add zeros to make the signal 64 samples long. (2) Use a 64 point DFT to find the frequency spectrum, which will consist of a 33 point real part and a 33 point imaginary part. (3) Pad the right side of the frequency spectrum